



Die neue Welt der Microservices und Container



Lernziele

- Sie haben einen ersten Überblick über die neue Welt der Microservices und Container.
- Sie haben Microservices in einer Kubernetes Umgebung zum laufen gebracht.

Zeitlicher Ablauf

- Container Life-Cycle
 - ◊ Die Sicht des Entwicklers (Dev)
 - ◊ Die Sicht des Operatings (Ops)
- Architekturmuster Microservices
 - ◊ Konzepte für Microservices
 - ◊ Applikation vs Microservices vs Container
- Alles Container oder was?
- Reflexion
- Lernzielkontrolle
- Bonus: Microservices und Teambildung, Domain Driven Design

Die Sicht der Entwickler (Dev)



ASP.NET Core-Webanwendung

Projektvorlagen zum Erstellen von ASP.NET Core-Anwendungen für Windows, Linux und macOS mithilfe von .NET Core oder .NET Framework. Erstellen Sie Web-Apps mit Razor Pages, MVC und Blazor oder Single-Page-Anwendungen (SPA) mit Angular, React oder React + Redux. Erstellen Sie auch Web-APIs, gRPC-Dienste und Workerdienste.

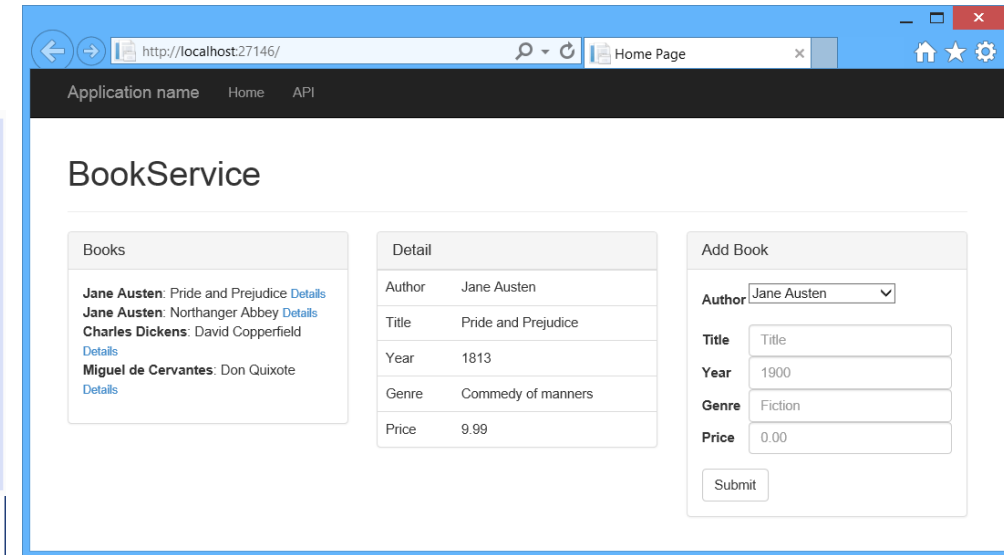
C#

Windows

Linux

macOS

Web



1



Webanwendung (Model-View-Controller)

Eine Projektvorlage zum Erstellen einer ASP.NET Core-Anwendung mit Beispielen für ASP.NET Core-MVC-Ansichten und -Controller. Diese Vorlage kann auch für RESTful HTTP-Dienste verwendet werden.



Razor-Klassenbibliothek

Eine Projektvorlage zum Erstellen einer Razor-Klassenbibliothek.



Angular

Eine Projektvorlage zum Erstellen einer ASP.NET Core-Anwendung mit Angular.



React.js

[Zusätzliche Projektvorlagen abrufen](#)

2

Erweitert

- ☒ Für HTTPS konfigurieren
- ☒ Docker-Unterstützung aktivieren

3

Docker Desktop installieren

Zum Debuggen in einem Container muss Docker Desktop installiert sein, und die Windows-Features für Container und Hyper-V müssen aktiviert werden. Dieser Vorgang erfordert erhöhte Zugriffsrechte und nimmt einige Minuten in Anspruch. Führen Sie nach Abschluss der Installation einen Neustart durch.

Möchten Sie Docker Desktop installieren und diese Features aktivieren?

☐ Diese Meldung nicht mehr anzeigen

Ja

Nein

About Docker Desktop

Discover Docker Enterprise Edition

Settings

Check for Updates

Diagnose and Switch to Win

Docker Hub

Documentati

Kitematic

Sign in / Crea

Repositories

Kubernetes

Restart...

Quit Dock

Veröffentlichungsziel auswählen

App Service

App Service Linux

Containerregistrierung

Azure Virtual Machi

IIS, FTP usw.

Ordner

Containerregistrierung

Registrierung zum Speichern und Bereitstellen von Docker

☐ Neue Azure Container Registry-Instanz erstellen

☐ Vorhandene Azure Container Registry-Instanz auswäh

☒ Docker-Hub

☐ Benutzerdefiniert

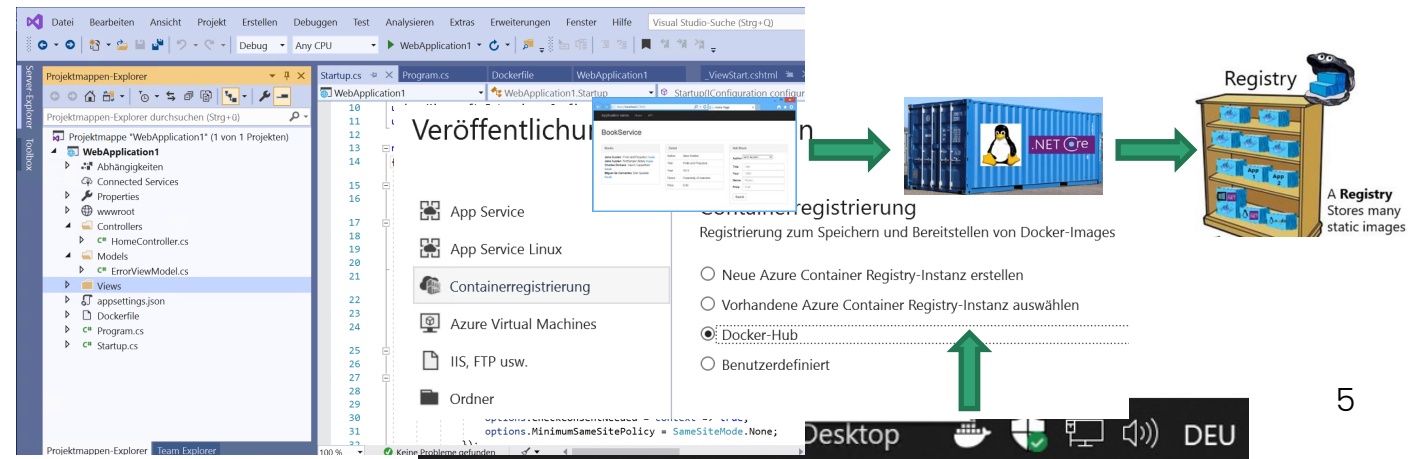
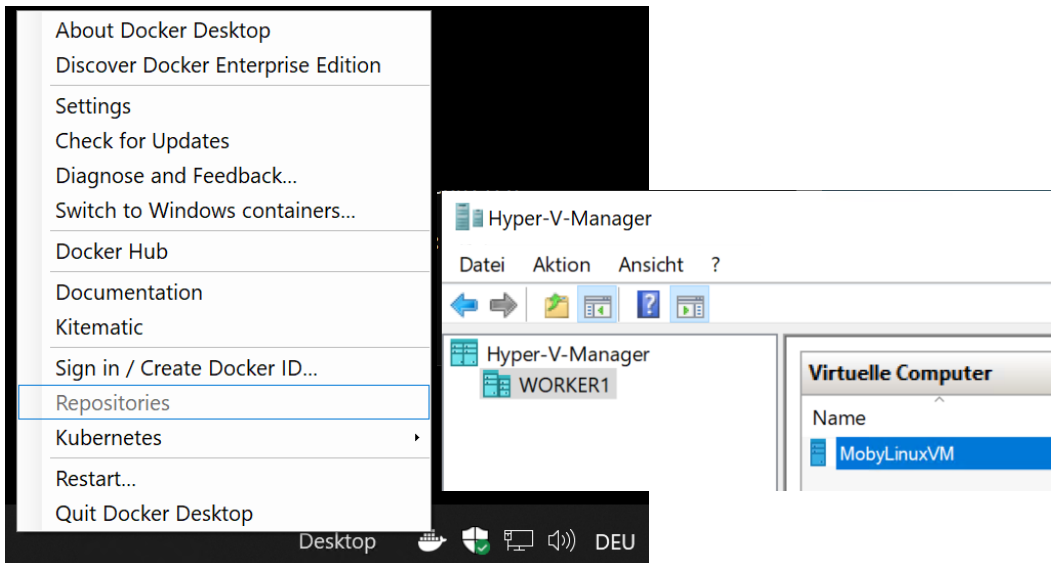
5



Was ist passiert auf Entwicklerseite (Dev)?

- 2 • Docker Desktop (Applikation und Service) wurde installiert (= Container Umgebung)
- 3 • Hyper-V (Virtualisierung) wurde aktiviert
- Eine Linux VM wurde erstellt

- 1 • Eine C# Webanwendung wurde erstellt
- Kompiliert in einem Container und mit allen Abhängigkeiten, als Container Image verpackt
- 4 • Das Container Image wurde in einem Container gestartet
- 5 • Das Container Image wurde in einer Registry (z.B. hub.docker.com) veröffentlicht.



Die Sicht des Operatings (Ops)

6

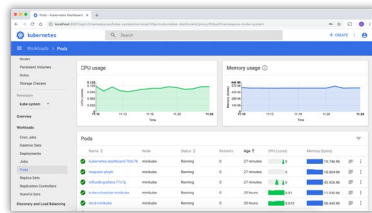
- Starten und Orchestrieren (Kubernetes) von Containern mittels

• Docker

- ◊ ~~docker run~~
- ◊ ~~docker compose~~

• Kubernetes

- ◊ (kubectl run ...)
- ◊ kubectl apply -f YAML
- ◊ Dashboard



- Infrastruktur z.B. Virtuelle Maschine(n)
- Mit Microservices/Container

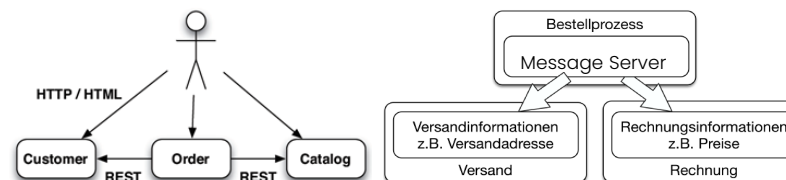


Es entfällt das Installieren von Applikationen!



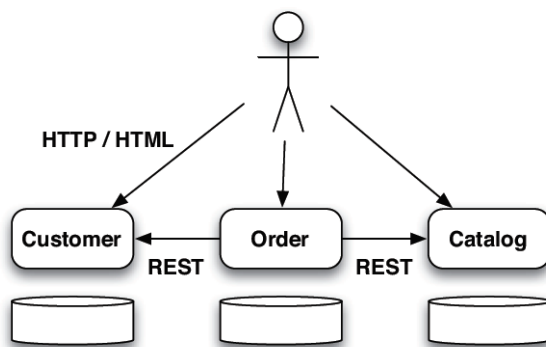
Was hat der Entwickler gemacht?

- Der Entwickler hat das Architekturmuster Microservices verwendet, um eine Webanwendung zu erstellen.
- Was sind Microservices?
 - ◇ Microservices – ein Ansatz zur Modularisierung von Software.
 - ◇ **Das Neue:** Microservices nutzen als Module einzelne Programme, die als eigene Prozesse laufen. Der Ansatz basiert auf der UNIX-Philosophie. Sie lässt sich auf drei Aspekte reduzieren:
 - Ein Programm soll nur eine Aufgabe erledigen, und das soll es gut machen.
 - Programme sollen zusammenarbeiten können.
 - Nutze eine universelle Schnittstelle.
 - ◇ In UNIX sind das Textströme: `cat adressen.txt | sort | uniq`
 - ◇ Bei Microservices das Internet (TCP/IP, REST oder Messages, Events)



Einfacher Webshop mit Microservices

- Der Webshop besteht aus drei Microservices: *Order*, *Customer* und *Catalog*
- Order kommuniziert mittels Catalog und Customer via REST. Ausserdem bietet jeder Microservice einige HTML-Seiten an.
- Zusätzlich ist ein vierter Microservices Webshop vorhanden, der dem Benutzer mit einem Menü den einfachen Einstieg in das System ermöglicht.



Auto Shop

Customer
Catalog
Order

Customer Service

List of customers

ID	Name
1	John Doe
2	Jane Smith
3	Bob Johnson

Catalog Service

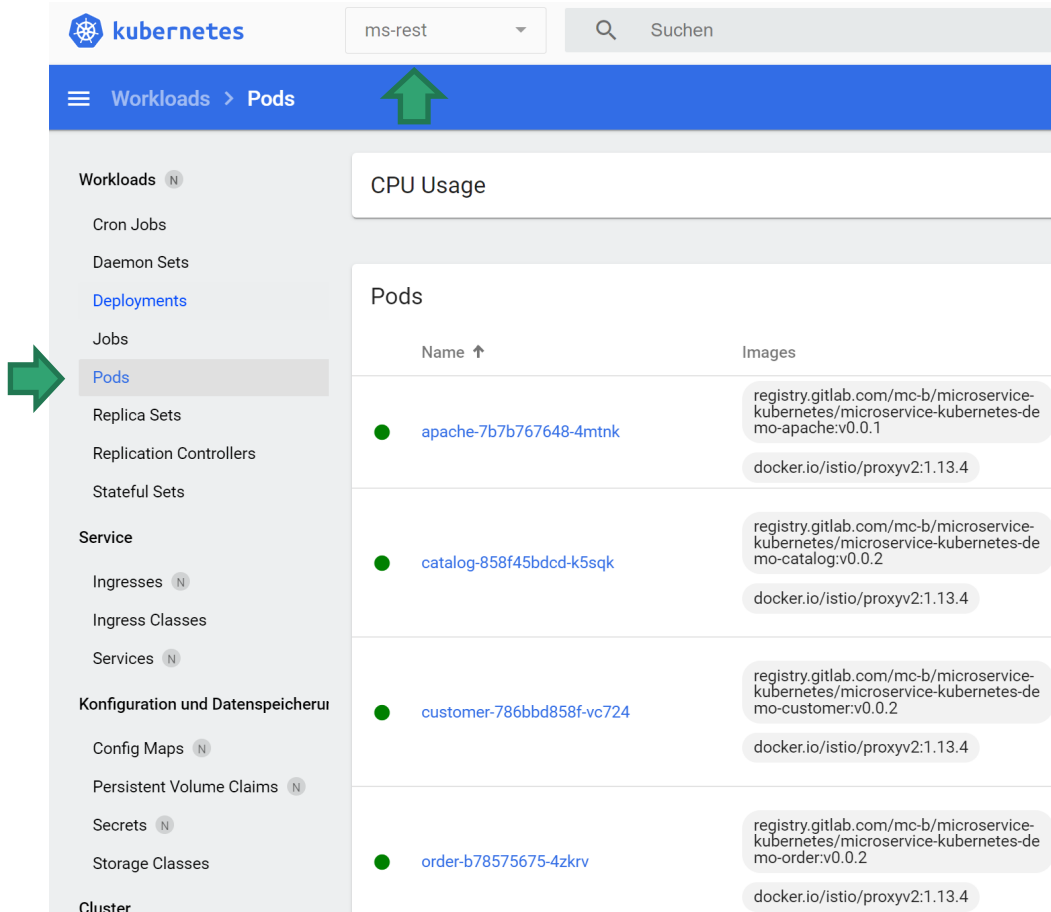
List of products

ID	Name
1	Land Rover Range Rover
2	Land Rover Discovery
3	Land Rover Defender
4	BMW 3er
5	BMW X5

Orders Service on order-d45f

Order ID	Customer Name	Product Name
1	John Doe	Land Rover Range Rover
2	Jane Smith	BMW 3er
3	Bob Johnson	Audi Q5

Tool für Operating: Kubernetes Dashboard

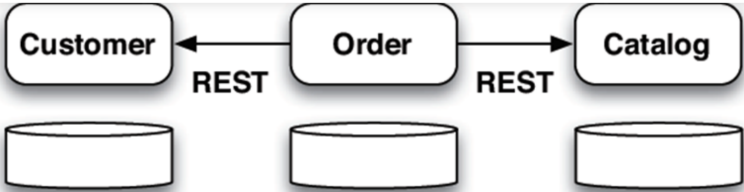


- Das Dashboard ist die webbasierte Kubernetes-Benutzeroberfläche.
- Sie können das Dashboard verwenden, um einen Überblick über die auf Ihrem Cluster ausgeführten Anwendungen zu erhalten und um einzelne Kubernetes-Ressourcen (wie Pods, Services usw.) zu erstellen oder zu ändern.
- **Für den Zugriff auf das Dashboard ist kein Token nötig, einfach "Überspringen" oder "Skip" drücken.**
- **Dann im Pulldown "ms-rest" auswählen und "Pods" oder "Workload" Anwählen**

Übung: Einfacher Webshop (REST)

jupyter Microservices-REST (unsaved changes)

File Edit View Insert Cell Kernel Help Not Trusted Python 3



Quelle: Buch Microservices Rezepte

Das Beispiel besteht aus drei Microservices: **Order**, **Customer** und **Catalog**.

Order nutzt **Catalog** und **Customer** mit der REST-Schnittstelle. Ausserdem bietet jeder Microservice einige HTML-Seiten an.

Zusätzlich ist im Beispiel ein Apache-Webserver installiert, der dem Benutzer mit einer Webseite einen einfachen Einstieg in das System ermöglicht.

Ebenso steht ein [Hystrix Dashboard](#) als eigener Kubernetes-Pod zur Verfügung. Das Beispiel nutzt die Java-Library Hystrix22, um Resilience zu erreichen. Diese Bibliothek führt Aufrufe in einem anderen Thread Pool aus und implementiert unter anderem einen Timeout für die Aufrufe.

```
In [ ]: ! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/apache.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/catalog.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/customer.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/hystrix.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/order.yaml
! kubectl apply -f https://raw.githubusercontent.com/mc-b/misegr/master/ewolff/ms-kubernetes/postgres.yaml
! kubectl config view -o=jsonpath='{.clusters[0].cluster.server }' | sed -e 's/https://http:/' -e "s/6443/${kubectl get service -n ms-rest --output=json | jq '.spec.ports[0].port'}"
```

- Kubernetes Namespace: **ms-rest**
- **Ergebnis:** Der Webshop lässt sich als ein Container (Übung 01-1) oder mehrere Service-Container betreiben. Läuft unverändert in unterschiedlichen Container-Umgebungen (z. B. Docker, Podman, Kubernetes). Zeigt, wie Microservices-Architektur, Containerisierung und DevOps-Prinzipien ineinandergreifen.
- **Erkenntnisse:** Container ermöglichen portables und reproduzierbares Deployment. Durch klare Service-Trennung entsteht hohe Flexibilität und Skalierbarkeit. Standards wie die Open Container Initiative (OCI) sichern Kompatibilität zwischen Plattformen.



Übung: Einfacher Webshop (docker-compose)

Microservices und docker-compose

Docker Compose ist ein leistungsfähiges Werkzeug, das die Orchestrierung von Multi-Container-Docker-Anwendungen vereinfacht.

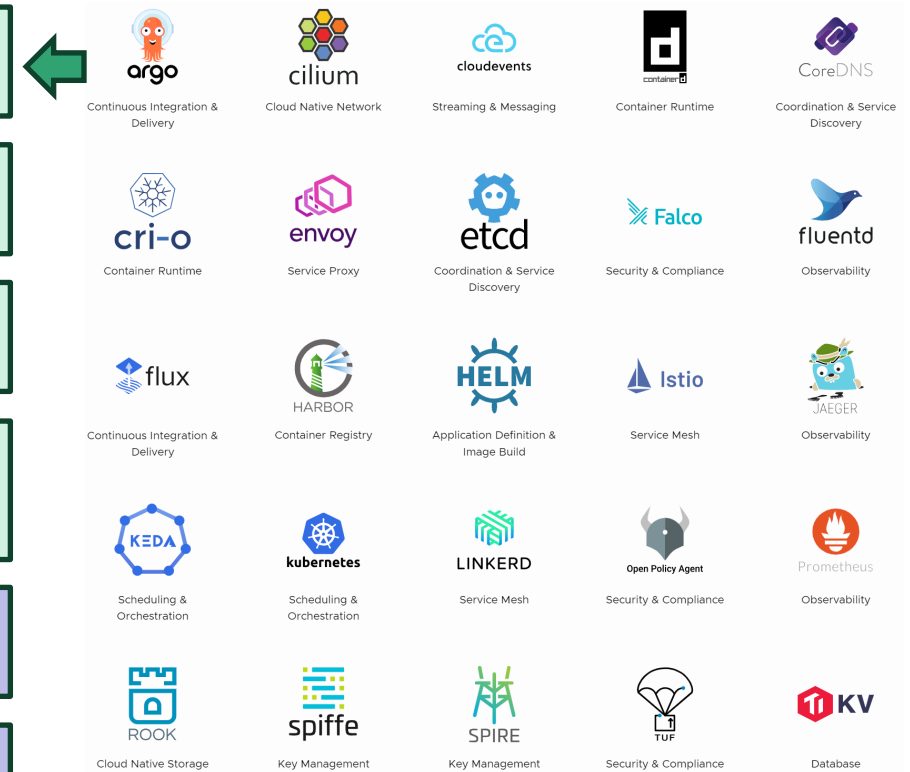
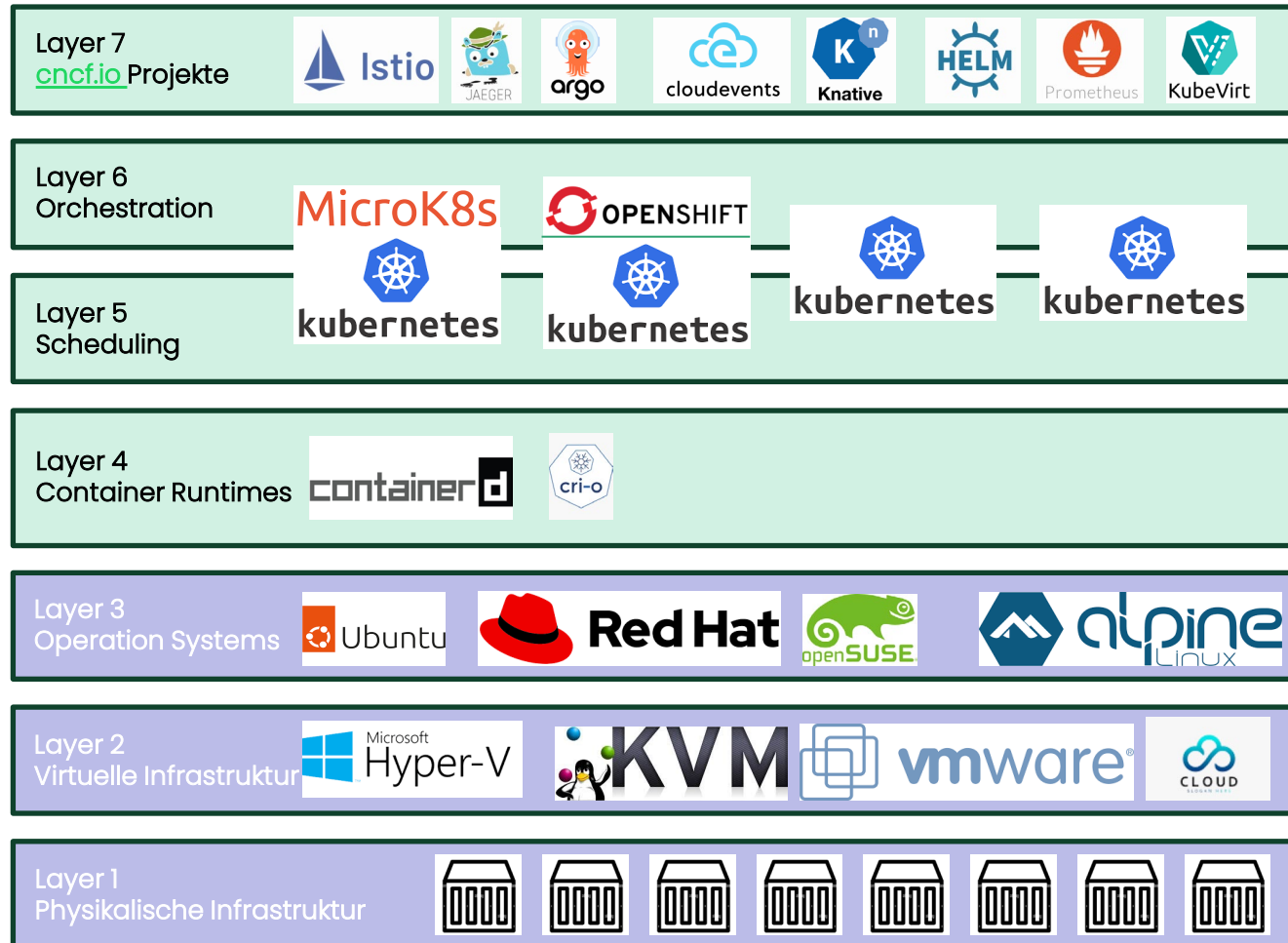
Anstatt mühsam einzelne Container manuell zu starten und zu konfigurieren, ermöglicht Docker Compose die Definition und Verwaltung von komplexen Anwendungen in einem einzigen YAML-Datei-Format.

Zuerst erstellen wir eine Docker Compose YAML-Datei:

```
] : %%bash
cat <<EOF% >docker-compose.yaml
services:
  customer:
    container_name: customer
    image: registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/customer:2.0.0
    ports:
      - "8080"
  catalog:
    container_name: catalog
    image: registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/catalog:2.0.0
    ports:
      - "8080"
  order:
    container_name: order
    image: registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/order:2.0.0
    ports:
      - "8080"
  webshop:
    container_name: shop
    image: registry.gitlab.com/ch-mc-b/autoshop-ms/app/shop/shop:2.0.0
    ports:
      - "8090:8080"
```

- Als Alternative zur Kubernetes
- Fragen:
 - ◇ Wie viele Instanzen wurden pro Container gestartet?
 - ◇ Wie wird das Port-Mapping durchgeführt?

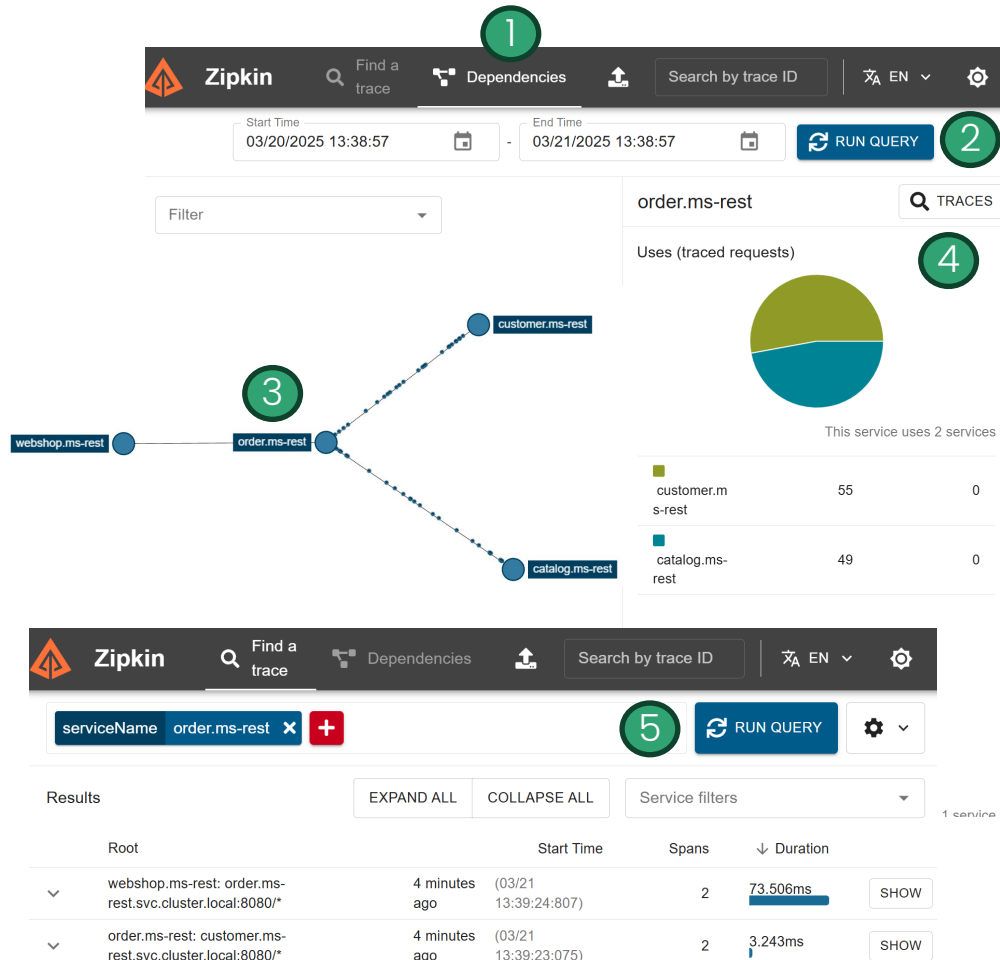
Layer 7 CNCF.io Projekte ab ca. 2022



<https://www.cncf.io/projects/>

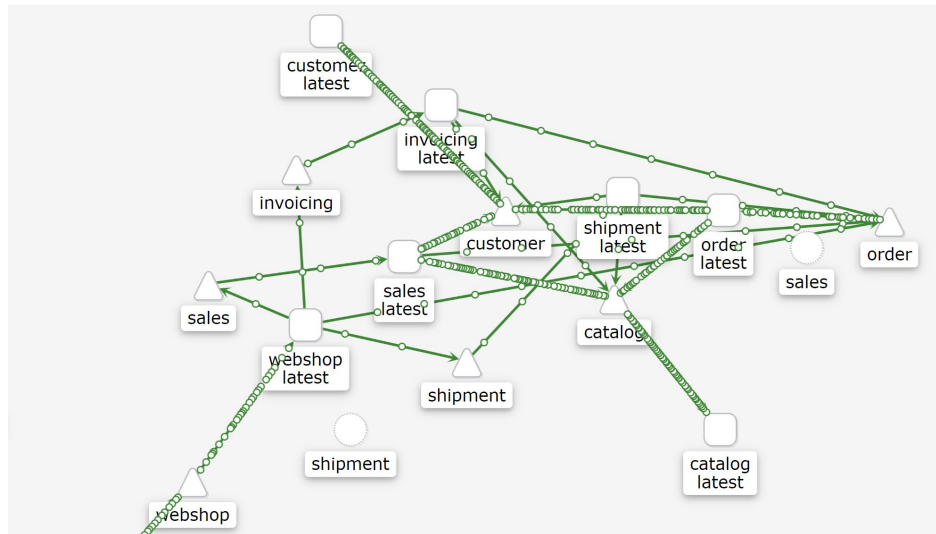
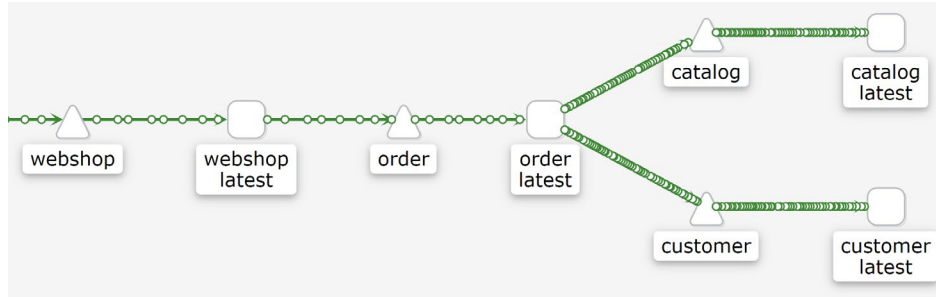
Die (vereinfachten) »Layer« der Container-Welt

Tools für Entwickler und Architekten – Istio+Zipkin



- **Istio** erhöht die Sicherheit durch Verschlüsselung der Kommunikation zwischen den Containern und liefert Telemetriedaten (Traffic) ohne Codeänderungen in den Anwendungen.
- **Zipkin** hilft beim Sammeln von Telemetriedaten, die zur Behebung von Latenzproblemen in Service-Mesh (Netzwerk von Microservices) erforderlich sind.
- Visualisierung und Anzeige Traces
 1. Wechsel auf Tab «Dependencies»
 2. Run Query anwählen
 3. order.ms-rest anwählen, TRACES wird eingeblendet
 4. TRACES anwählen, Wechsel auf Tab «Find a trace»
 5. Run Query – die Antwortzeiten des Microservices werden angezeigt

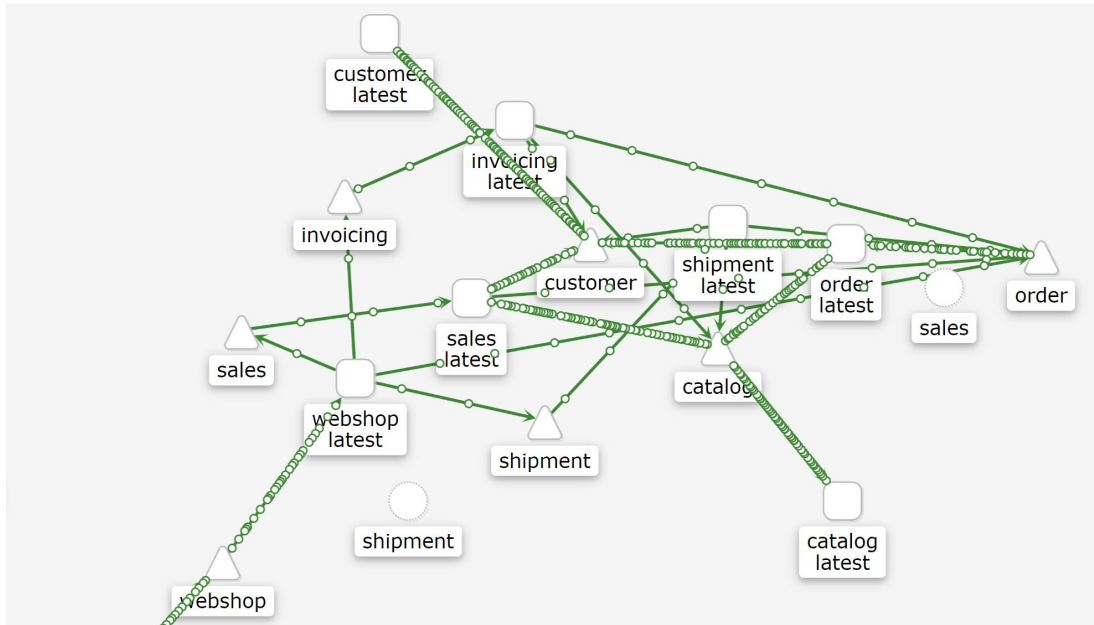
Begriff: Service Mesh vs. Big Ball of Mud



- Der Begriff Service-Mesh beschreibt das Netzwerk von Microservices und die Wechselwirkungen zwischen ihnen.
- Ein Service-Mesh organisiert und verwaltet die Kommunikation zwischen Microservices, wodurch klare Strukturen entstehen, und die Wartbarkeit verbessert wird.
- Im Gegensatz dazu führt eine fehlende Organisation oft zu einem "Big Ball of Mud" (dt.: grosse Matschkugel, grosses Knäuel), einer chaotischen und schwer wartbaren Softwarearchitektur.

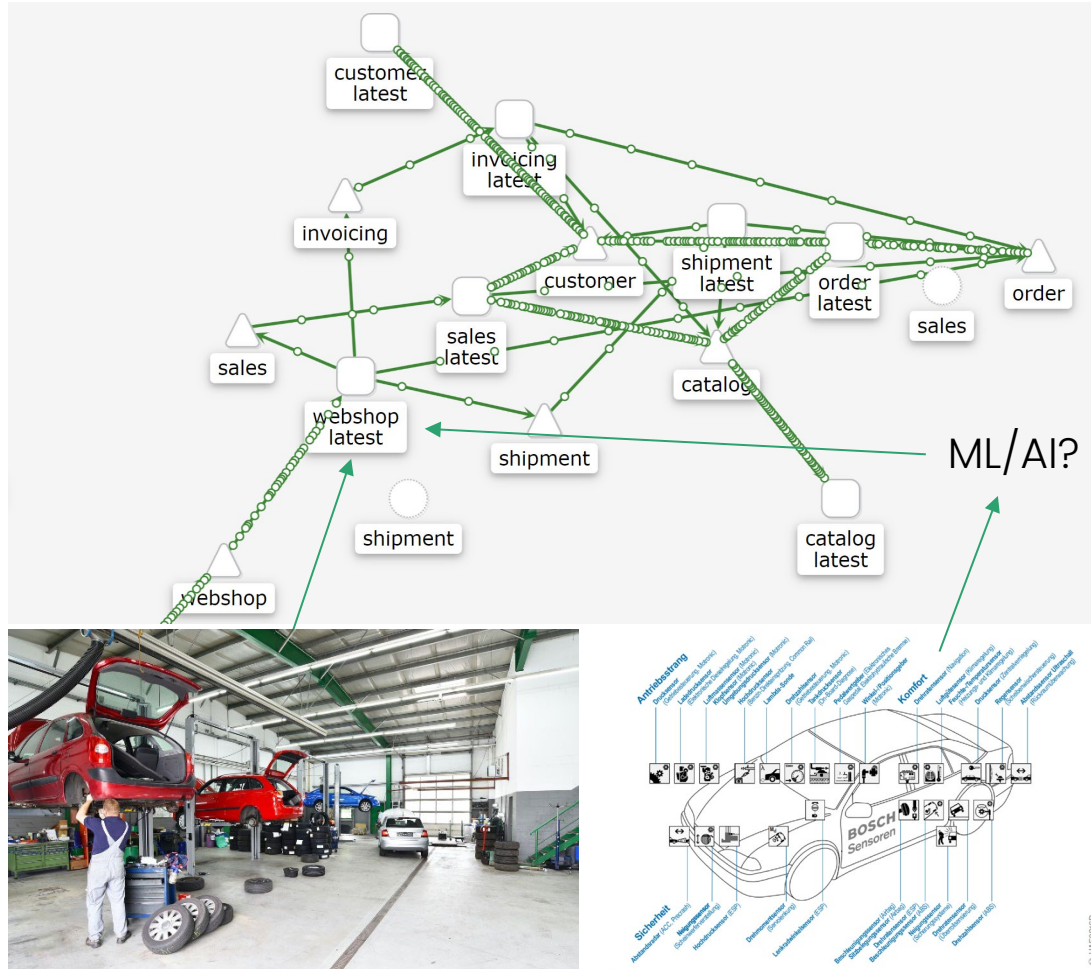


Übung: Big Ball of Mud (BBoM)



- Kubernetes Namespace: **ms-bbum**
- Das Beispiel besteht aus Microservices: **Order, Customer, Catalog, Shipment, Invoicing** und **Sales**
- **Order** nutzt **Catalog** und **Customer** mittels deren REST-Schnittstelle. **Shipment, Invoicing** und **Sales** holen Daten von **Order, Catalog** und **Customer** mittels deren REST-Schnittstelle.

Wie weiter, wenn der Kunde weitere Anforderungen hat?



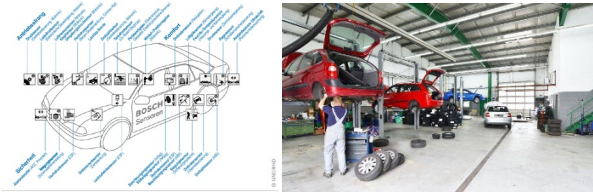
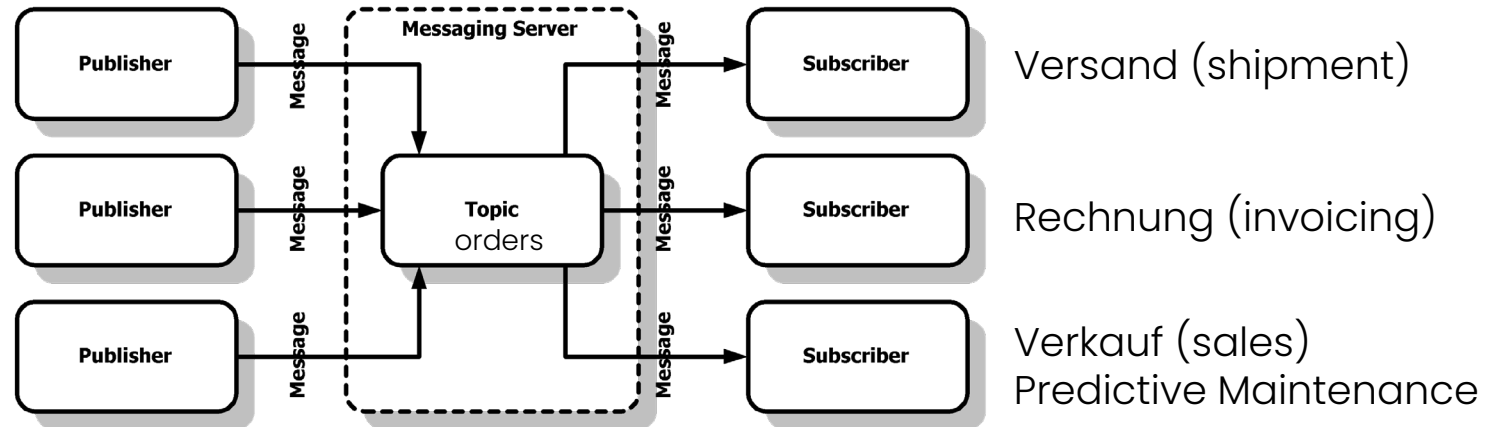
- Obwohl unsere Software-Architektur bereits einiges an Übersichtlichkeit verloren hat, kommen immer neue Anforderungen des Kunden, u.a.:
 - ◇ Einbindung der Autoreparaturgarage
 - ◇ Nutzen der Sensoren in den Autos, z.B. für Predictive Maintenance (Vorausschauende Wartung)
- Gartner sagt:
 - ◇ Wer Microservices, Container, Kubernetes einsetzt, wird neue Features **80 Prozent schneller** als ihre Mitbewerber implementieren können.

Lösung: Big Ball of Mud → asynchroner Kommunikation

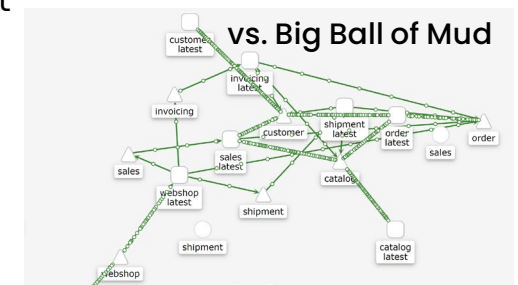
- Die Überleitung von einem Big Ball of Mud zu einer Architektur, die asynchrone Kommunikation nutzt, führt zur Verbesserung von Softwaresystemen.
- Anstatt einer unstrukturierten und schwer wartbaren Codebasis können durch die Implementierung asynchroner Kommunikationsmuster verschiedene Vorteile realisiert werden.
 - ◇ Beim Ausfall eines Kommunikationspartners wird die Nachricht später übertragen, wenn der Kommunikationspartner wieder verfügbar ist.
 - ◇ Die Übertragung und auch die Bearbeitung einer Nachricht (Message) können fast immer garantiert werden.
 - ◇ Asynchrone Microservices können Messages als Events implementieren. Events bieten eine **fachliche Entkopplung**. Ein Event könnte beispielsweise Bestellung eingegangen sein. Jeder Microservice kann selbst entscheiden, wie er auf den Event reagiert.

Asynchrone Microservices: Publish/Subscribe Muster

Bestellprozess
(order, customer, catalog)



- Bei Asynchronen Microservices, kann das Publish/Subscribe Muster verwendet werden.
- Es ersetzt die Punkt-zu-Punkt-Verbindungen durch einen zentralen Server (Messaging Server), zu dem sich Datenproduzenten (Publisher) und -nutzer (Subscriber) gleichermassen verbinden können
- Dadurch bleiben die Abonnenten (Subscriber) unabhängig vom Sender (Publisher).
- Die Abstimmung erfolgt mittels Topics. Topics können, wie Verzeichnisse betrachtet werden, wo die Messages als einzelne Dateien abgestellt werden.

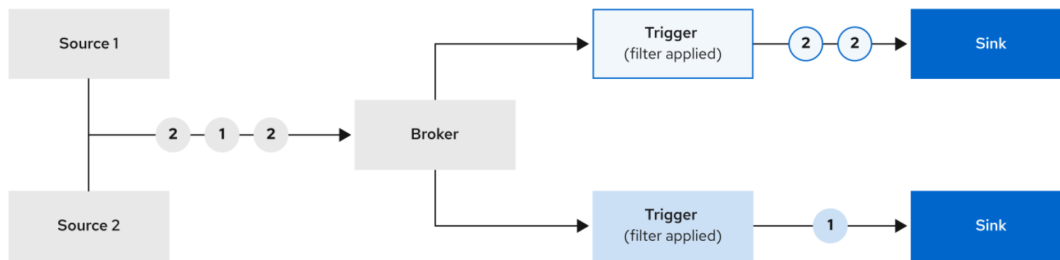




Übung: Asynchrone Microservices (Messaging)

Übung: Broker und Filter mit K-native

● ○ ● Events



113_OpenShift_0920

Quelle: K-native

Broker Ein Broker in Knative ist eine zentrale Komponente, die als Verteiler von Ereignissen (Events) fungiert. Er empfängt Ereignisse von verschiedenen Quellen und sorgt dafür, dass diese an die richtigen Abonnenten weitergeleitet werden. Ein Broker besteht aus zwei Hauptteilen:

- Ingress: Hier kommen die Ereignisse an.
- Channel: Ein internes Kommunikationssystem, das die Ereignisse an die Abonnenten weiterleitet.

Filter (Trigger) Ein Filter, auch Trigger genannt, definiert eine Regel, die bestimmt, welche Ereignisse an eine bestimmte Knative-Services oder Endpunkte gesendet werden. Triggers filtern die Ereignisse basierend auf bestimmten Kriterien, wie z.B. dem Ereignistyp oder anderen Attributen, und leiten sie dann an den entsprechenden Empfänger weiter.

Zusammenspiel von Broker und Trigger

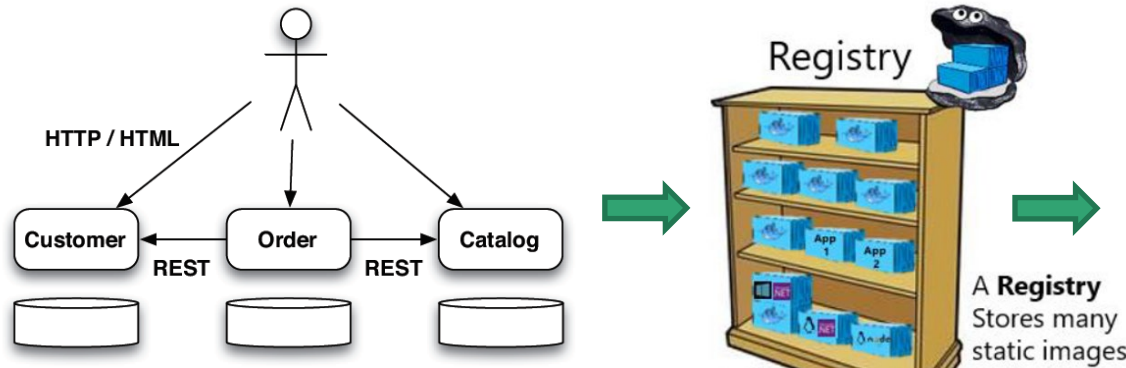
- Ereignisaufnahme: Der Broker empfängt Ereignisse von verschiedenen Quellen.
- Verteilung: Der Broker verteilt die Ereignisse an die entsprechenden Triggers.
- Filterung: Triggers filtern die Ereignisse gemäß den definierten Regeln.
- Zustellung: Gefilterte Ereignisse werden an die entsprechenden Services oder Endpunkte zugestellt.

- Kubernetes Namespace: **ms-brkr**
- **K-native** (Kubernetes-native) ist eine leistungsfähige und flexible Plattform für die Bereitstellung von serverlosen Anwendungen auf Kubernetes.
- Entwickelt von Google und der Open-Source-Community, ermöglicht K-native Entwicklern, sich auf das Schreiben von Code zu konzentrieren, ohne sich um die zugrunde liegende Infrastruktur kümmern zu müssen.
- Durch die Nutzung der bestehenden Kubernetes-Ökosystems bietet K-native
 - ◇ automatische Skalierung
 - ◇ einfache Bereitstellung und Verwaltung von Containern nach dem Serverless (FAAS) Muster
 - ◇ Unterstützung für **ereignisgesteuerte Architekturen**

Applikation vs Microservices vs Container

Entwickler (Dev)

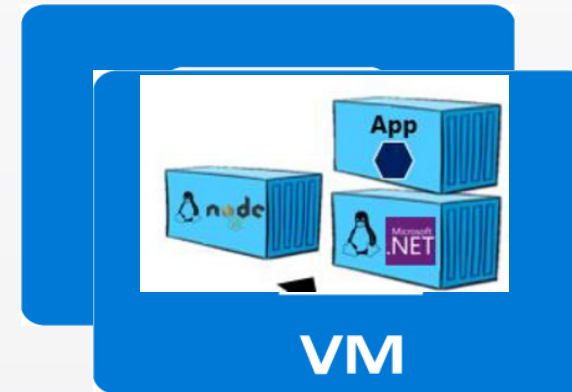
- Microservices



- 1ne Applikation = 1:n Microservices
- 1 Microservice = 1:n Container Image Versionen
- 1 Microservice = 1:n Container Instanzen

Operating (Ops)

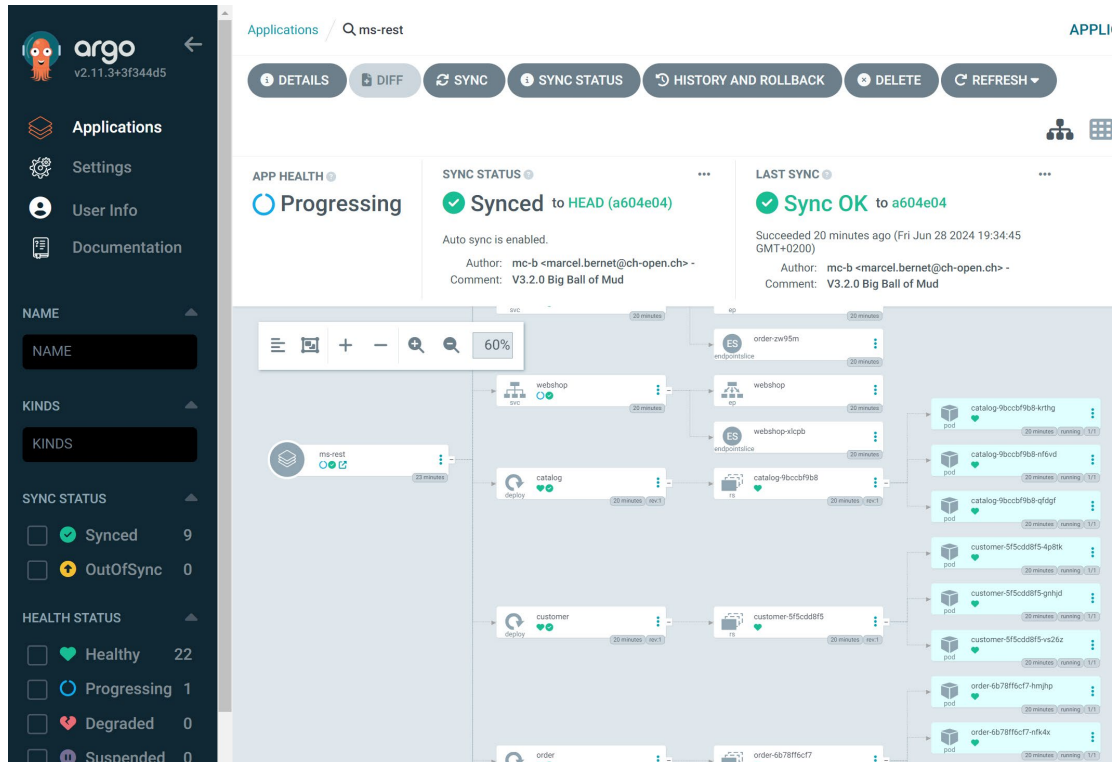
- VMs als Behälter für Container



- Container basieren auf [Container Images](#)
- 1 Container Image = 1:n Container Instanzen
- 1 Container Instanz = 1 Prozess

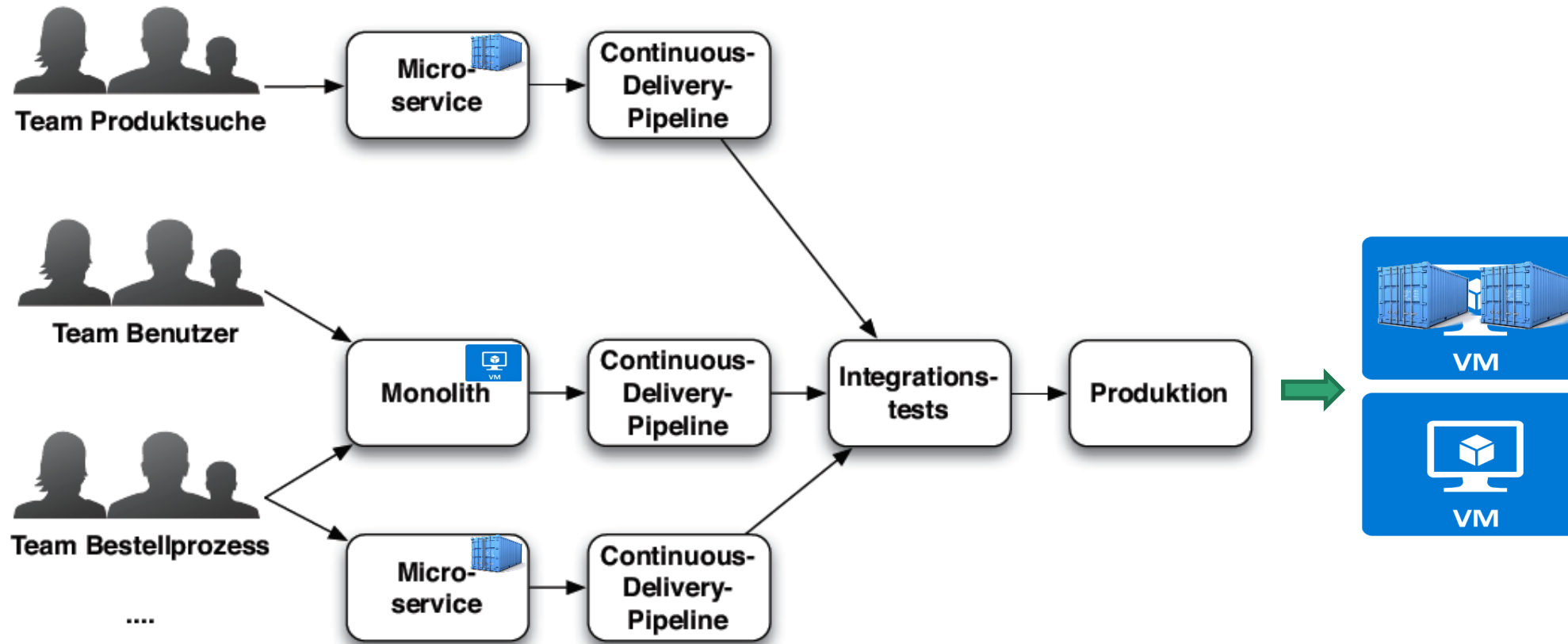


Übung: Applikationen deployen (ArgoCD)

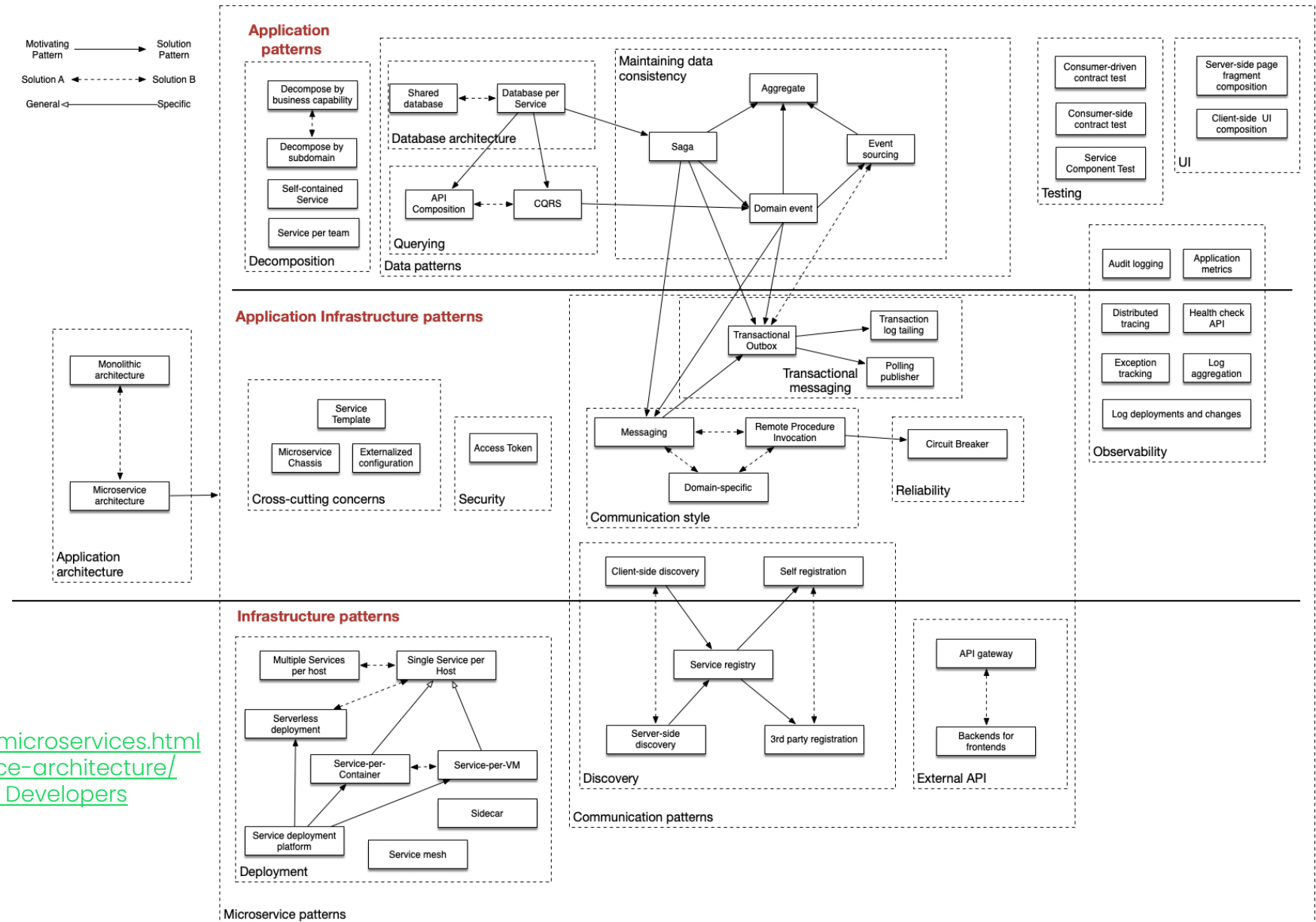
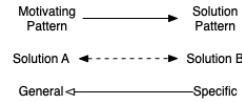


- **GitOps** ist ein modernes Paradigma für die Verwaltung und Bereitstellung von Infrastruktur und Anwendungen. Es basiert auf der Idee, dass das gesamte System, einschließlich der Anwendungs- und Infrastrukturkonfiguration, in einem Git-Repository versioniert und verwaltet wird.
- **ArgoCD** ist ein Kubernetes-nativer Continuous Deployment (CD) Tool, das speziell für GitOps entwickelt wurde. Es überwacht Git-Repositories auf Änderungen und synchronisiert den gewünschten Zustand der Kubernetes-Cluster automatisch mit den in den Repositories definierten Konfigurationen.
- ArgoCD ist nicht dafür ausgelegt, den Netzwerk-verkehr sichtbar zu machen oder zu überwachen.

Alles Container oder was?



Patterns (Muster)



<http://microservices.io/patterns/microservices.html>
<https://eng.uber.com/microservice-architecture/>
 7 Microservices Best Practices for Developers



Auto Shop Beispiele und Übungen

- Die Auto Shop Beispiele und Übungen zeigen den Weg von einer Monolithische Applikation über Microservices bis Serverless.
 - ◇ [Monolithische Applikation](#)
 - ◇ [Einfacher WebShop implementiert als Microservice-Applikation](#)



Reflexion

- Container und Microservices verändern die IT-Landschaft Technologisch und Organisatorisch (DevOps).
- Container und Microservices Beschleunigen den Time-to-Market von neuen Produkten.
- Container und Microservices brauchen eine effiziente Continuous Integration / Continuous Delivery (CI/CD) Pipeline.
- In der Entwicklung reden wir von Microservices meinen aber modulare Applikationen in Container Images verpackt und betrieben als Container.
- Microservices und die Wechselwirkungen zwischen ihnen, werden oft als Service-Mesh bezeichnet.



Lernzielkontrolle

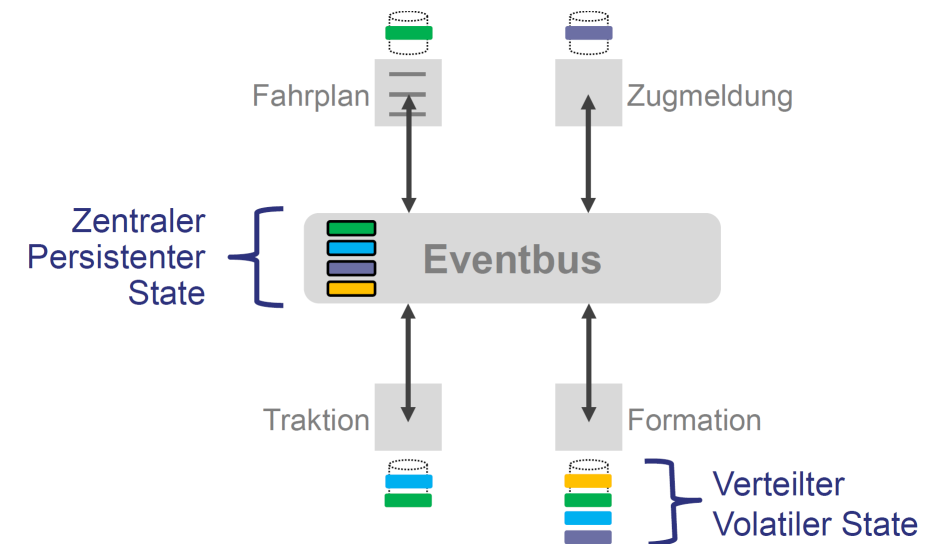
- Sie haben einen ersten Überblick über die neue Welt der Container und Microservices.
- Sie haben Microservices in einer Kubernetes Umgebung zum laufen gebracht.

Domain Driven Design (DDD)

- DDD ist ein Satz von Werkzeugen, die beim Entwerfen und Implementieren von hochwertiger Software helfen.
- *Ihre Organisation kann nicht in allen Bereichen exzellent sein, deshalb sollte sie umsichtig wählen, womit sie herausstechen will.*
- Die **strategischen Werkzeuge** von DDD helfen, die aus Wettbewerbssicht besten Entwurfs- und Integrationsentscheidungen für Ihr Geschäft zu treffen.
- Die **taktischen Werkzeuge** von DDD können unterstützen, nützliche Software zu entwerfen, die das für den Geschäftserfolg notwendige Vorgehen exakt modelliert.

Unser Unternehmen und Microservices

- Was passiert im Hintergrund?
 - ◇ Statt monolithischer Applikationen – entkoppeln der Dienste, und einzeln als ein **Microservice**, in je einem Container betrieben und vernetzt.
 - ◇ Dies ermöglicht eine massive Skalierbarkeit, Verfügbarkeit und Portierbarkeit.
 - ◇ Bietet die Möglichkeit durch Modularisierung und sukzessive Aktualisierung Big Bang Releases zu entschärfen.

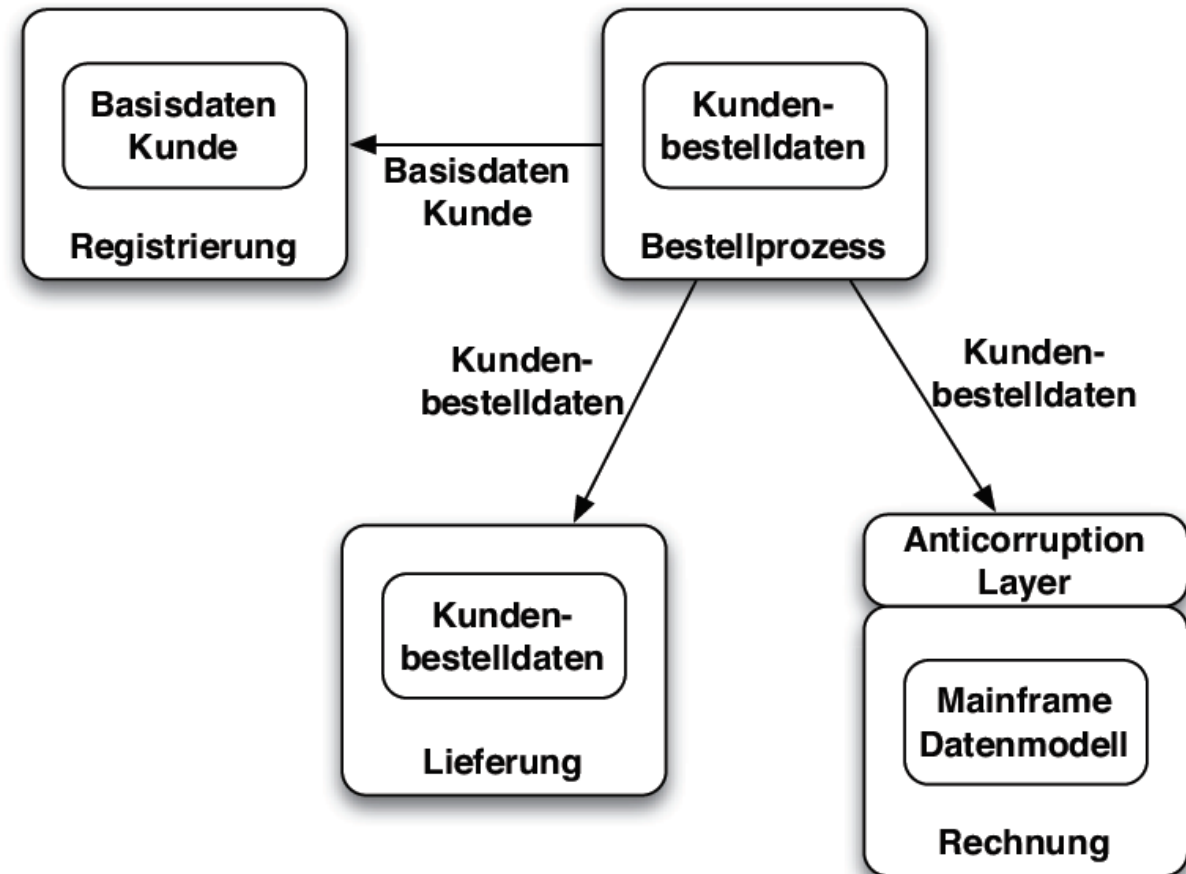


Bounded Context (dt.: begrenzter Kontext)



Context Mapping (dt.: Abbilden von Kontexten)

- Abbildung Integration von Bounded Contexen



Arten von Mapping

- **Partnership (dt.: Partnerschaft)** – Koordination zweier Teams mit abhängigen Zielen aber eigenem Bounded Context.
- **Shared Kernel (dt.: geteilter Kern)** – Zwei (oder mehr) Teams, die sich ein kleines, aber gemeinsames Modell teilen.
- **Customer-Supplier (dt.: Kunde-Lieferant)** – Lieferant stellt Kunde Modell zur Verfügung.
- **Conformist-Beziehung (dt.: Konformist, Mitläufer)** – Team übernimmt Modell von Partner, weil das Modell bereits gut eingeführt ist.
- **Anticorruption Layer (dt.: Antikorruptionsschicht, Antiverfälschungsschicht)** – Team baut eine Übersetzungsschicht.
- **Open Host Service (dt.: offen angebotener Dienst)** – [ProgrammableWeb](#)
- **Published Language (dt.: veröffentlichte Sprache)** – Wohldokumentierte Sprache zum Informationsaustausch, z.B. <http://www.ech.ch/xmlns/>.



Übung

- Betrachten Sie ein Ihnen bekanntes Projekt
 - ◇ Wie sieht die Teamstruktur aus?
 - Ist sie technisch oder fachlich getrieben
 - Muss die Struktur für einen Microservice-Ansatz geändert werden?
 - Wie muss sie geändert werden?
 - ◇ Ist die Architektur auf Teams sinnvoll verteilbar? Schliesslich soll jedes Team unabhängige fachliche Komponenten verantworten und dort Features umsetzen können.
 - Welche Änderungen an der Architektur wären notwendig?
 - Wie aufwendig sind die Änderungen?